

Simone Lidonnici - 2061343

Pietro Adamo Morelli - 2087359

Alberto Vona - 2068261



The issue with "Streaming Perception"

When talking about **streaming perception**, we are directly addressing one of the greatest challenges in real-time applications, such as autonomous driving.

Traditional perception models must be *incredibly fast* to be safe, but there is a fundamental problem: as soon as they finish processing an image, the world around them *has already changed*.

This temporal misalignment can lead to critical errors, because the vehicle makes decisions based on **outdated information**.

StreamYOLO addresses this problem by introducing a new metric, **sAP**, which forces the model not only to be accurate, but also to be incredibly fast, explicitly considering its own processing time.

The Intuition of StreamYOLO

Predicting the next frame: Since current real-time detectors are sufficiently fast, the processing delay creates a fixed mismatch between the processed frame and the real-world state. The key solution proposed by the paper is therefore to train the model to predict the results of the next frame, starting from the current state.

Training via triplets: The architecture is trained using data triplets composed of the previous frame, the current frame, and the ground truth of the next frame.

Two fundamental modules: To allow the detector to predict the future, the paper introduces two specific modules: Dual Flow Perception (**DFP**) and Trend Aware Loss (**TAL**).

Dual Flow Perception (DFP) Module

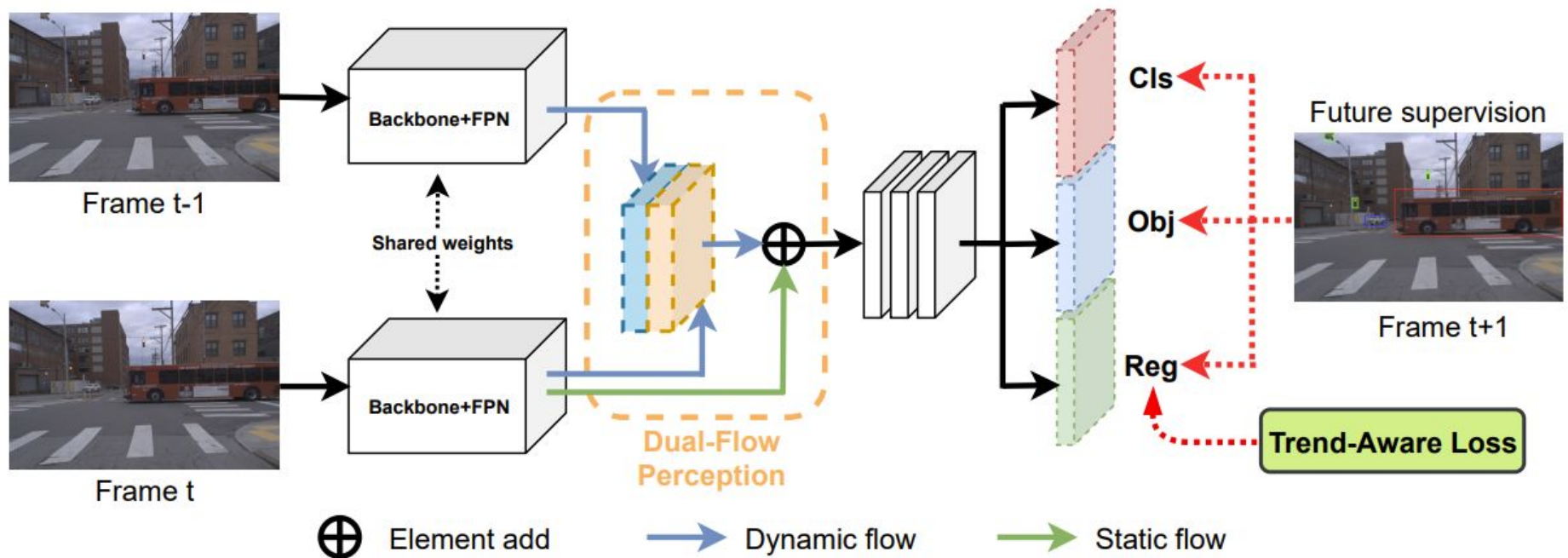
So, how does the Dual Flow Perception module work? Its task is to fuse the feature maps extracted from the current and previous frames, providing the model with a rich understanding of *motion*.

Specifically, a **dynamic flow** combines these temporal features to capture motion trends.

In parallel, a **static flow**, thanks to a residual connection, provides the crucial base features for accurately localizing and classifying objects within the current frame.

This combined flow ensures that the model understands both **what is in the frame** and **how it is moving**.

Dual Flow Perception (DFP) Module



The Mathematics of Trend-Aware Loss (TAL)

To refine our predictions, we need to account for the fact that objects within a single frame move at very different speeds.

Intuitively, **fast-moving** objects are much harder to predict than **static ones**, so we should pay them more attention during training.

This is where the Trend-Aware Loss (TAL) comes in: it *adaptively* assigns a higher weight to the loss of faster objects.

We measure this 'moving trend' by calculating the Intersection over Union (**IoU**) between boxes in consecutive frames.

A low **IoU** indicates a high-speed **displacement**.

The Mathematics of Trend-Aware Loss (TAL)

To handle this mathematically, we define a trend factor ω_i for each object:

$$mIoU_i = \max_j (\{IoU(box_i^{t+1}, box_j^t)\})$$

$$\begin{cases} \omega_i = \frac{1}{mIoU_i} & mIoU_i \geq \tau \\ \omega_i = \frac{1}{\nu} & mIoU_i < \tau \end{cases}$$

Here, τ acts as a threshold to identify 'new' objects that don't have a match in the previous frame, while ν is a constant used to calibrate the weight for these cases.

By fine-tuning these two parameters, we can balance the model's focus between tracking existing movement and recognizing new appearing entities.

Introduction to our project

Inspired by the paper, we tried to implement the same prediction behaviour for *Pokemon tracking*. We used **yolov5** as a base to our implementation, specifically the **yolov5s** model.



Dataset versions

The dataset used was created in three iterations:

1. **Dataset micro** (~ 350 images): images labeled by ourselves, to create a first model that predicts still images.
2. **Dataset single** ($\sim 4.5K$ images): using the detection of the micro version a bigger dataset was created and a final model for still images was trained.
3. **Dataset stream** ($\sim 10.5K$ images): using the previous version we labeled entire videos to create the dataset for the model that predict the next frame.

Changes to Dataloader

The **dataloader** was modified to process pairs of images and labels. Specifically, at any given time t , the model receives a *6-channel* input image created by concatenating the images from frames t and $t-1$.

Additionally, the **dataloader** takes the labels corresponding to frames t and $t+1$, which are utilized to compute the weights of the loss function.



Implementation of DFP

We inserted the **DFP** after the backbone of the model to take the feature of the pair of images in input and execute a convolution to fuse the features.

After the convolution the *channels are halved* to make the output of the **DFP** usable for the rest of the model.

Training the model

For the **training phase**, we first developed a Pokémon recognition model trained on static images, used as a baseline for our final predictive model. In this final model, we disabled all geometric image *augmentations* and *transformations*, keeping only the color augmentations.

```
hsv_h: 0.015  
hsv_s: 0.7  
hsv_v: 0.4
```

TAL: τ and ν

In our implementation the *trend factor* of each object is calculated only between object of the same class. This ensures there aren't wrong association between **boxes** with **high IoU** but that correspond to different Pokémon.

τ	ν	AP_{all}	AP_{50}	AP_{75}	AP_{small}	AP_{medium}	AP_{large}	AR_{max1}	AR_{max10}	AR_{max100}	AR_{small}	AR_{medium}	AR_{large}
0.3	1	0.1424	0.3229	0.1072	0.1176	0.1055	0.1451	0.2092	0.2159	0.2159	0.1199	0.1710	0.2228
0.3	1.5	0.1285	0.3072	0.0807	0.0993	0.1076	0.1216	0.2026	0.2101	0.2101	0.1064	0.1808	0.2101
0.3	2	0.1295	0.2978	0.0920	0.1238	0.0969	0.1245	0.1946	0.2011	0.2011	0.1343	0.1531	0.2122
0.4	1	0.1493	0.3307	0.1144	0.1133	0.1188	0.1408	0.2192	0.2270	0.2270	0.1167	0.1887	0.2154
0.4	1.5	0.1474	0.3352	0.1150	0.1353	0.1118	0.1442	0.2179	0.2241	0.2241	0.1412	0.1772	0.2276
0.4	2	0.1512	0.3415	0.1098	0.1213	0.1184	0.1448	0.2207	0.2272	0.2272	0.1247	0.1823	0.2288
0.5	1	0.1453	0.3287	0.1070	0.1125	0.1092	0.1457	0.2149	0.2217	0.2217	0.1153	0.1766	0.2278
0.5	1.5	0.1468	0.3232	0.1166	0.1119	0.1172	0.1420	0.2130	0.2180	0.2180	0.1127	0.1766	0.2111
0.5	2	0.1471	0.3273	0.1165	0.1232	0.1185	0.1399	0.2200	0.2274	0.2274	0.1260	0.1895	0.2189





THANKS
FOR
WATCHING!